

Daisy 架構、流程及使用 簡介

2025/1

Daisy 的特色

1. 支援分層結構(更適合智慧城市的需求, 可分散流量)
2. 實現 handover 的功能, 使用者可以簡單的使用
3. 可以傳送json形式的資料給client
4. 有許多流程相關的可自訂義區塊(包含strategy、server logic、client logic、metric handler)

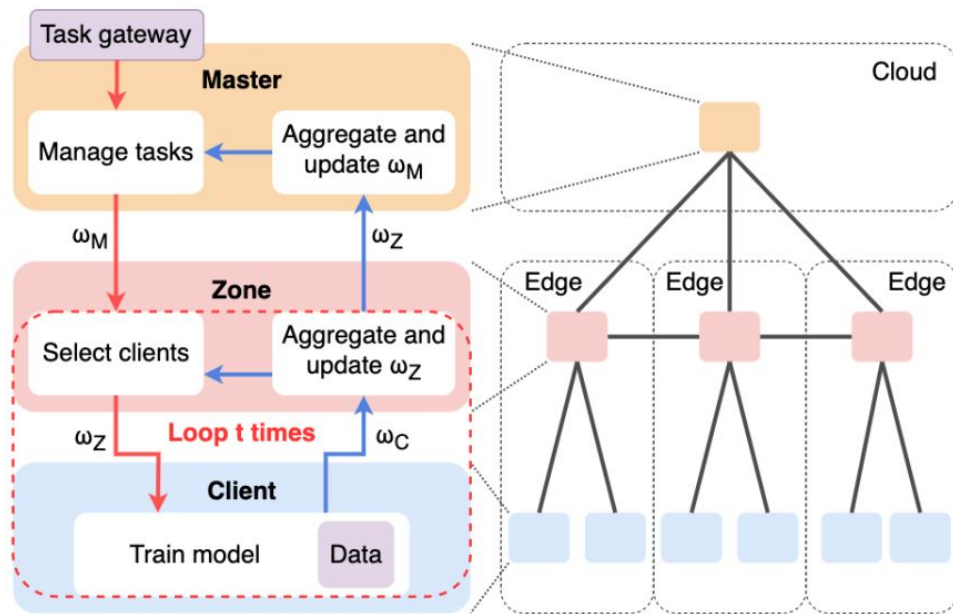
Daisy 的架構

master node: cloud 中的 server, child 可以是 zone node 或 client

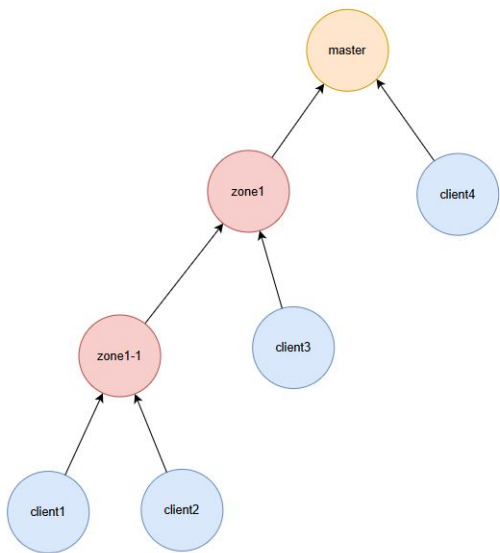
task gateway: master node 的一部分, 負責以 http 接收任務

zone node: 在 edge 的 server, zone 之間在做 handover 時可以互相通訊, child 可以是 zone node 或 client

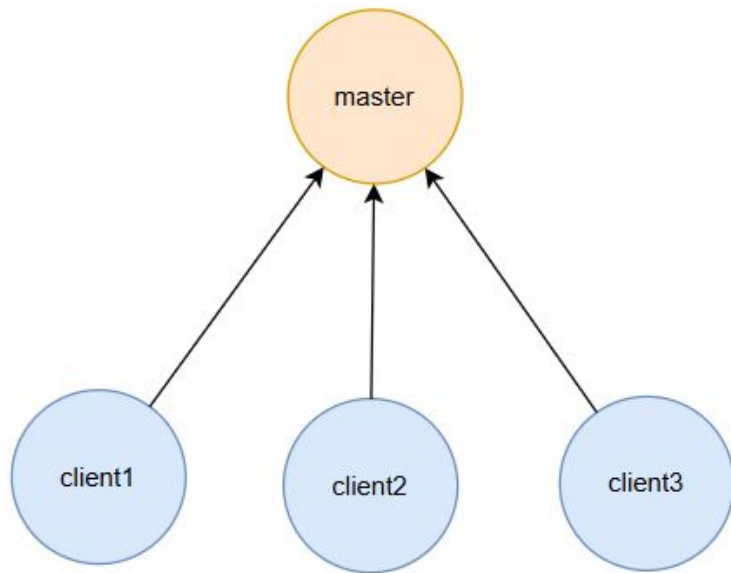
client: 負責使用 dataset 訓練 model, 接著將 model 上傳 parent



node 架構示例



zone node 可以有同樣是 zone node 的 child



master node 可以直接接 client, 形成單層結構



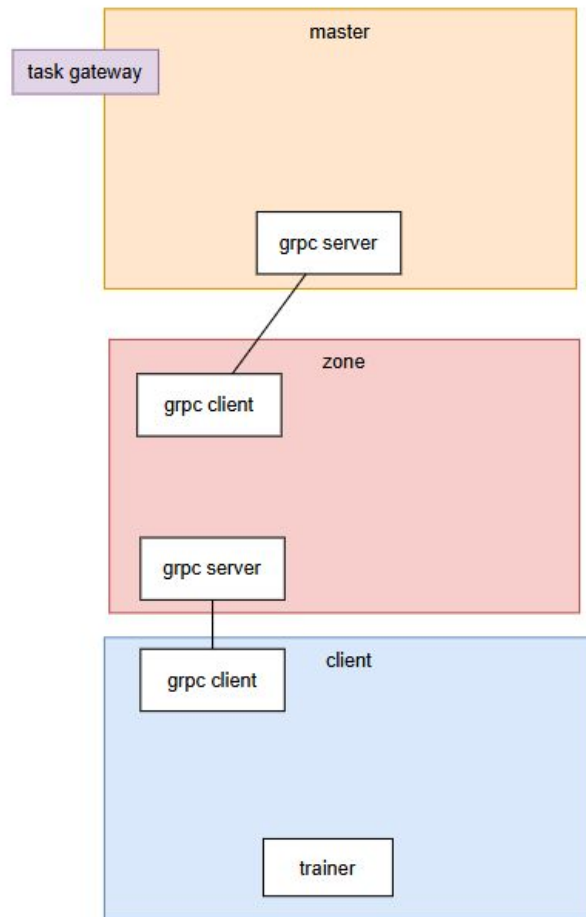
Daisy 的運作流程



1. 建立node

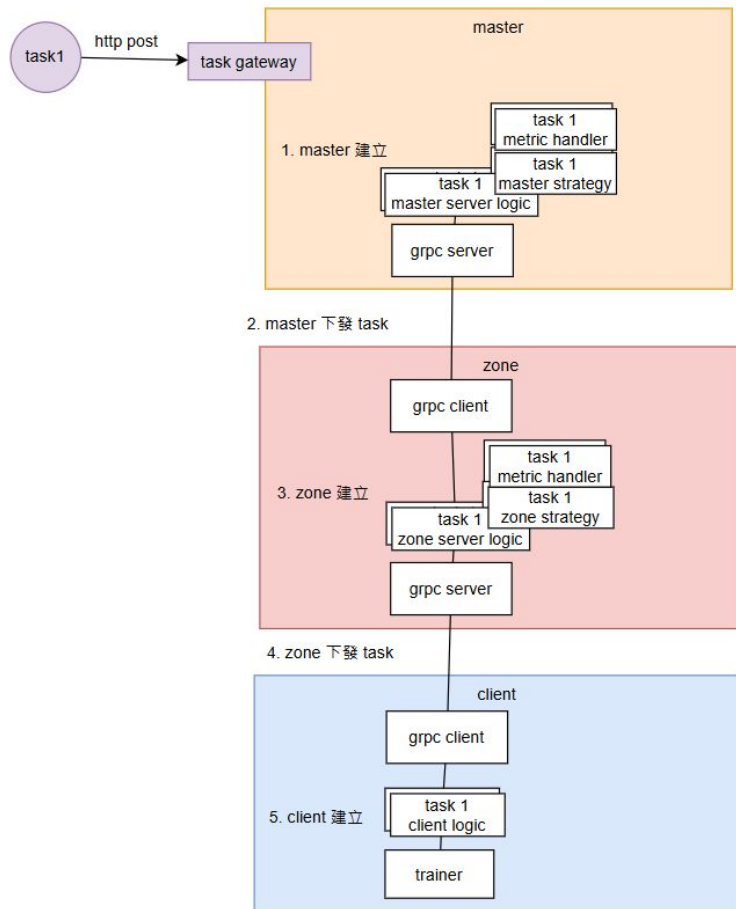
node 在建立後會根據提供的ip、port、parent address 去建立 server 以及連接 parent node

client 進行訓練時使用什麼model、dataset 等，在建立時便已固定，會是一個trainer 的 object



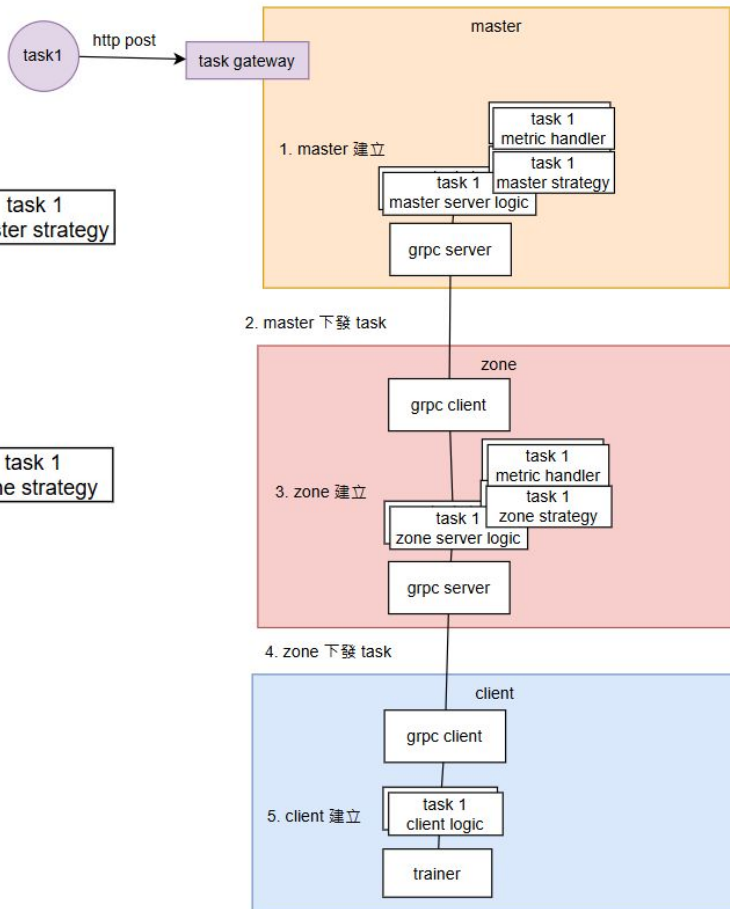
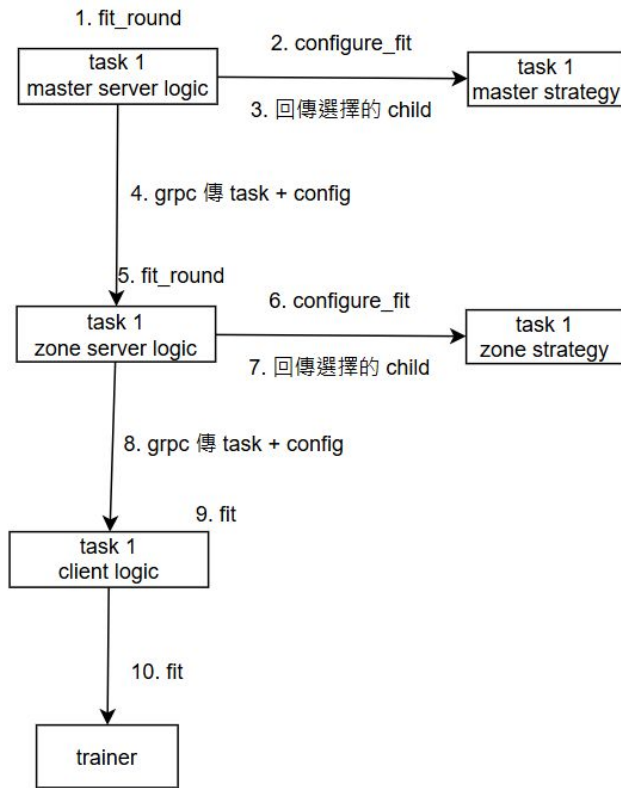
2. 發布 task

1. task 透過 http 從 master node (task gateway) 傳入
2. node 跟據 task 內的描述建立 task 指定的 logic、strategy (每個 task 都會分別創建)
3. 跟據 logic 的執行結果將task 傳給 child 或是讓 trainer 進行訓練

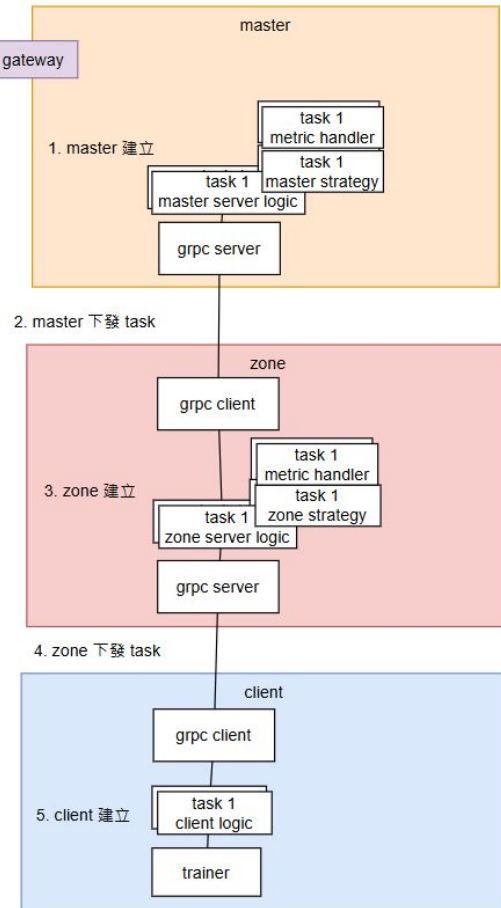
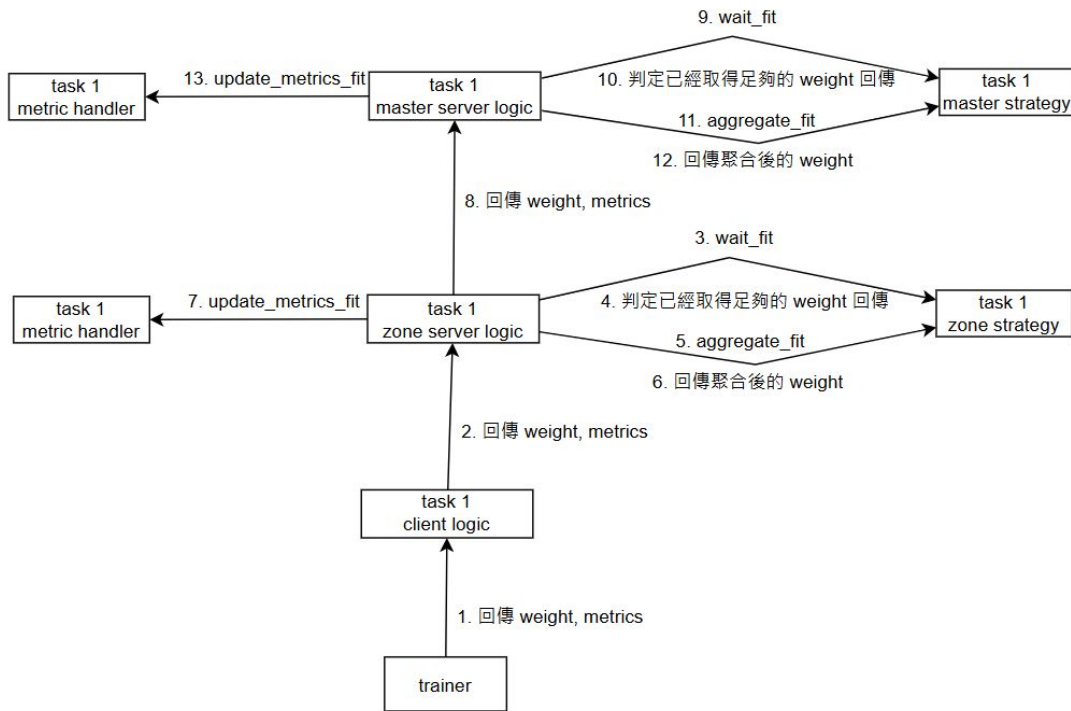


2. 發布 task (細部流程)

logic、operator 的建立
是在執行途中透過
dynamic import 完成的



3. 回傳



回傳頻率

假設

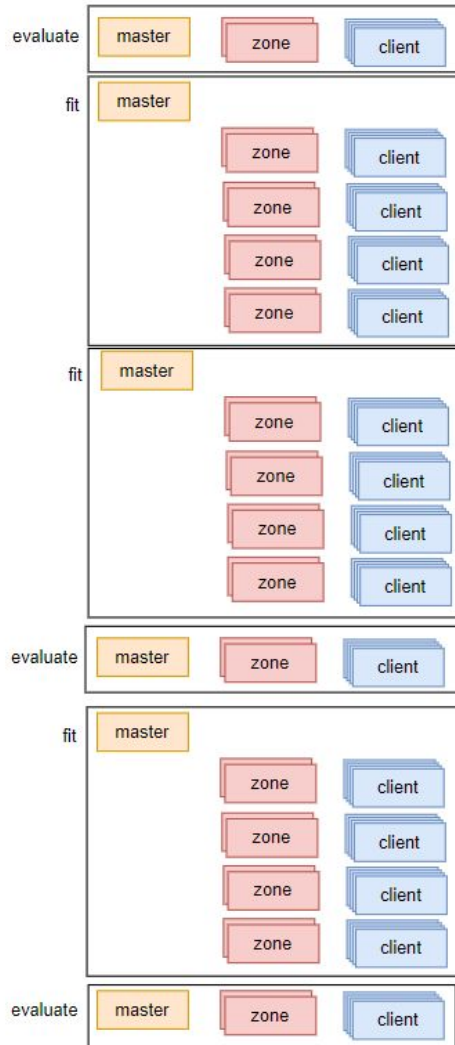
NUM_ROUNDS = 3

ZONE_COMM_FREQUENCY=4

EVALUATE_INTERVAL=2

zone 會在聚合 ZONE_COMM_FREQUENCY 次後回傳

evaluate 只有 Master 會發起





Daisy 客製化 example



可客製化 class

- **Server Logic:**
Master、zone 的處理邏輯 (加解密、與 client 預先溝通)
- **Client Logic:**
Client 的處理邏輯 (加解密、接收 server 的其他訊息)
- **Strategy:**
決定發任務給哪些 child、檢查是否要開始聚合、聚合演算法
- **MetricsHandler:**
處理 accuracy、loss 等 metric
- **Trainer:**
client 啟動時作為 parameter, model + dataset 都在此

Server Logic

決定 master/zone 的運作邏輯, 通常會去call strategy 與 metrics_handler 的 function 使用

daisyfl/operator/base/base_server_logic.py
內有基本示例

daisyfl/operator/server_logic.py

```
39     self.communicator: Communicator = communicator
40     self.strategy: Strategy = strategy
41     self.metrics_handler: MetricsHandler = metrics_handler
```

```
43     @abstractmethod
44     def fit_round(
45         self,
46         parameters: Parameters,
47         task: Task,
48     ) -> Optional[
49         Tuple[Optional[Parameters], Optional[Report]]
50     ]:
51         """Perform a single round fit."""
```

```
53     @abstractmethod
54     def evaluate_round(
55         self,
56         parameters: Parameters,
57         task: Task,
58     ) -> Optional[Report]:
59         """Validate current global model."""
```

Client Logic

決定 client 的運作邏輯，通常會去 call trainer
進行 fit 或 evaluate

daisyfl/operator/base/base_client_logic.py
內有基本示例

daisyfl/operator/client_logic.py

```
28         self.trainer = trainer
```

```
32     @abstractmethod
33     def fit(
34         self, ins: FitIns,
35     ) -> FitRes:
```

```
38     @abstractmethod
39     def evaluate(
40         self, ins: EvaluateIns,
41     ) -> EvaluateRes:
```

Strategy

決定發任務給哪些child、檢查是否要開始聚合、聚合演算法, 由server logic 使用這些功能

daisyfl/strategy 內有 fedavg、fedasync、sec_agg 等基本示例

```
43 @abstractmethod
44 def configure_fit(
45     self, parameters: Parameters, config: Dict, **kwargs
46 ) -> List[Tuple[ClientProxy, FitIns]]:
47     """Configure the next round of fitting."""
48
49 ~
50 @abstractmethod
51 def configure_evaluate(
52     self, parameters: Parameters, config: Dict, **kwargs
53 ) -> List[Tuple[ClientProxy, EvaluateIns]]:
54     """Configure the next round of evaluating."""
```

daisyfl/strategy/strategy.py

```
55 @abstractmethod
56 def wait_fit(
57     self, subtask_status: SubtaskStatus, **kwargs
58 ) -> bool:
59     """Wait for the termination condition of fitting."""
60
61 @abstractmethod
62 def wait_evaluate(
63     self, subtask_status: SubtaskStatus, **kwargs
64 ) -> bool:
65     """Wait for the termination condition of evaluating."""
```

```
67 @abstractmethod
68 def aggregate_fit(
69     self,
70     results: List[Tuple[ClientProxy, FitRes]],
71     **kwargs,
72 ) -> Tuple[Optional[Parameters], Dict]:
73     """Aggregate fitting results."""
74
75 @abstractmethod
76 def aggregate_evaluate(
77     self,
78     results: List[Tuple[ClientProxy, EvaluateRes]],
79     **kwargs,
80 ) -> Dict:
81     """Aggregate evaluation results."""
```

Metrics Handler

負責整理 fit 或 evaluate 後的 metrics 結果，並轉換成可以從 task gateway 用 http get 取得的結果

daisyfl/metrics_handler 內有基本示例

daisyfl/metrics_handler/metrics_handler.py

```
22 class MetricsHandler(ABC):
23     """Abstract base class for coping with metrics update and exposure."""
24     ~
25     @abstractmethod
26     def update_metrics_fit(self, config: Dict):
27         """Update fitting metrics."""
28
29     @abstractmethod
30     def update_metrics_evaluate(self, config: Dict):
31         """Update evaluation metrics."""
32     ~
33     @abstractmethod
34     def get_metrics(self,):
35         """Expose metrics through API gateway."""
36
37     @abstractmethod
38     def __repr__(self) -> str:
39         """Print metrics."""
```


Trainer

專門進行 model 的 fit 或 evaluate, 需要在 client 啟動時決定

examples/01_quickstart_pytorch/client.py 內有基本示例

examples/01_quickstart_pytorch/client.py

```
75 class DaisyTrainer(fl.client.NumPyTrainer):
76     def get_parameters(self, config):
77         return [val.cpu().numpy() for _, val in net.state_dict().items()]
78
79     def set_parameters(self, parameters):
80         params_dict = zip(net.state_dict().keys(), parameters)
81         state_dict = OrderedDict({k: torch.tensor(v) for k, v in params_dict})
82         net.load_state_dict(state_dict, strict=True)
83
84     def fit(self, parameters, config):
85         self.set_parameters(parameters)
86         train(net, trainloader, epochs=1)
87         config[METRICS] = { DATA_SAMPLES: len(trainloader.dataset) }
88         return self.get_parameters(config={}), config
89
90     def evaluate(self, parameters, config):
91         self.set_parameters(parameters)
92         loss, accuracy = test(net, testloader)
93         config[METRICS] = {
94             ACCURACY: accuracy,
95             LOSS: loss,
96             DATA_SAMPLES: len(testloader.dataset)
97         }
98         return config
99
104 if __name__ == '__main__':
105     # Start Daisy client
106     fl.client.start_client_numpy(
107         server_address=args.server_address,
108         parent_address=args.parent_address,
109         numpy_trainer=DaisyTrainer(),
110         metadata=metadata,
111     )
112
```

1. 建立 node

master 與 zone node 與 flower 相似

提供 ip + port 即可啟動

zone 需要額外提供 parent address

master 則需要額外提供建立task
gateway (api gateway) 的 address

examples/02_hierarchical_pytorch/master.py

```
18 fl.master.start_master(  
19     server_address=args.server_address,  
20     api_ip=args.api_ip,  
21     api_port=int(args.api_port),  
22 )
```

examples/02_hierarchical_pytorch/zone.py

```
10 fl.zone.start_zone(  
11     parent_address=args.parent_address,  
12     server_address=args.server_address,  
13 )
```

1. 建立 node

client 需要實作讓 client logic call 的
trainer

examples/02_hierarchical_pytorch/client.py

```
75 class DaisyTrainer(fl.client.NumPyTrainer):
76     def get_parameters(self, config):
77         return [val.cpu().numpy() for _, val in net.state_dict().items()]
78
79     def set_parameters(self, parameters):
80         params_dict = zip(net.state_dict().keys(), parameters)
81         state_dict = OrderedDict({k: torch.tensor(v) for k, v in params_dict})
82         net.load_state_dict(state_dict, strict=True)
83
84     def fit(self, parameters, config):
85         self.set_parameters(parameters)
86         train(net, trainloader, epochs=1)
87         config[METRICS] = { DATA_SAMPLES: len(trainloader.dataset) }
88         return self.get_parameters(config={}), config
89
90     def evaluate(self, parameters, config):
91         self.set_parameters(parameters)
92         loss, accuracy = test(net, testloader)
93         config[METRICS] = {
94             ACCURACY: accuracy,
95             LOSS: loss,
96             DATA_SAMPLES: len(testloader.dataset)
97         }
98         return config
99
104 if __name__ == '__main__':
105     # Start Daisy client
106     fl.client.start_client_numpy(
107         server_address=args.server_address,
108         parent_address=args.parent_address,
109         numpy_trainer=DaisyTrainer(),
110         metadata=metadata,
111     )
112
```

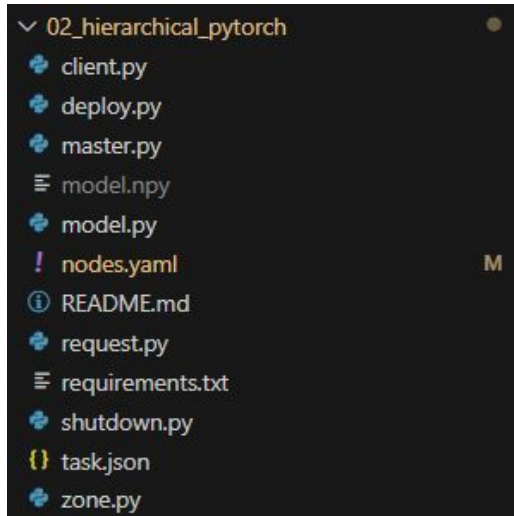
1. 建立 node

以 examples/02_hierarchical_pytorch 為例

可以通過 `python deploy.py -yaml nodes.yaml --init_model` 快速啟動並建立一個 random 的 `init_model`

`deploy.py` 內通過 `os.system` 執行 `master.py`、`zone.py`、`client.py` 等檔案

`master.py`、`zone.py`、`client.py` 內根據參數分別啟動 master node、zone node、client



2. 發布 task

通過 http post 將 task 的 json 傳至 task gateway 中, 可以在這裡指定 server logic、client logic、strategy、metrics handler 等可客製化 class 的 import 路徑

doc/tutorial/01_quickstart_in_pytorch.md 內有 key 所對應的意義

examples/02_hierarchical_pytorch/task.json

```
{
  "TID": "customized_fl_algorithm_demo",
  "NUM_ROUNDS": 2,
  "ZONE_COMM_FREQUENCY": 2,
  "EVALUATE_INTERVAL": 1,
  "EVALUATE_INIT_MODEL_MASTER": true,
  "MIN_WAITING_TIME_MASTER": 10,
  "MIN_WAITING_TIME_ZONE": 5,
  "MODEL_PATH": "model.npy",
  "OPERATORS": {
    "MASTER_SERVER_OPERATOR": ["my_operator.server_logic", "MyServerLogic"],
    "ZONE_SERVER_OPERATOR": ["my_operator.server_logic", "MyServerLogic"],
    "CLIENT_OPERATOR": ["my_operator.client_logic", "MyClientLogic"]
  },
  "STRATEGIES": {
    "MASTER_STRATEGY": ["my_strategy.fedavg", "MyFedAvg"],
    "ZONE_STRATEGY": ["my_strategy.fedavg", "MyFedAvg"]
  },
  "METRICS_HANDLERS": {
    "MASTER_METRICS_HANDLER": ["my_metrics_handler.hier_metrics_handler", "MyMetricsHandler"],
    "ZONE_METRICS_HANDLER": ["my_metrics_handler.hier_metrics_handler", "MyMetricsHandler"]
  },
  "SAVE_MODEL": true
}
```

2. 發布 task

examples/02_hierarchical_pytorch

以 examples/02_hierarchical_pytorch 為例

可以通過 `python request.py --json=task.json --api=http://127.0.0.1:9887/publish_task` 發布 task，因為 master node 在啟動時 api ip 設定為 0.0.0.0、api port 設定為 9887

request.py 內用 `requests.post`，傳送 json 至 task gateway

▼ 02_hierarchical_pytorch

client.py

deploy.py

master.py

model.npy

model.py

! nodes.yaml

① README.md

request.py

requirements.txt

shutdown.py

{ } task.json

zone.py

3. 等待執行結束

task 被完成後, http post 會接收到 response

3. 優雅的結束 node 的執行

以 `examples/03_customized_algorithms_pytorch` 為例

可以通過 `python shutdown.py` 關閉 master、zone、client

`shutdown.py` 會連接到 gRPC server (client 有一個專門負責 shutdown 的 gRPC server), 並傳送 shutdown message

command line

安裝 daisy

```
./dev/bootstrap.sh
```

以 examples/03_customized_algorithms_pytorch 為例

```
pip install -r requirements.txt
```

```
python deploy.py --yaml nodes.yaml --init_model
```

```
python request.py --json=task.json --api=http://127.0.0.1:9887/publish_task
```

```
python shutdown.py
```

如何進行 handover

以 examples/05_proximal_transmission_pytorch 為例

本 example 內 strategy 會隨機選擇 client 添加
config[TRANSITION] = True, client logic 遇到會執行
self.handover_fn("ip:port"), 執行後 daisy 內會進行
handover 的處理

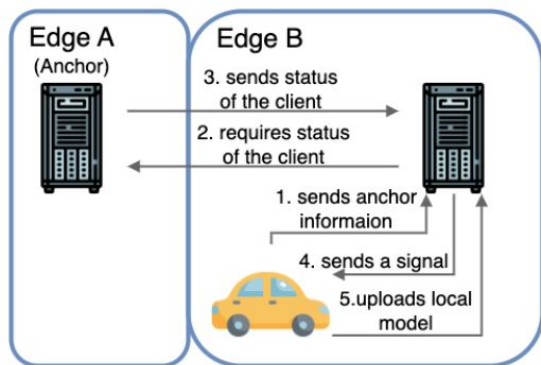


Fig. 4: The steps of edge roaming mechanism in Daisy

examples/05_proximal_transmission_pytorch/mcs_operator/client_logic.py

```
27 class MCSClientLogic(ClientLogic):
28     """Daisy Client operational logic to demonstrate the roaming mechanism."""
29
30     def __init__(self, trainer, get_anchor_fn: Callable, handover_fn: Callable,) -> None:
31         self.trainer = trainer
32         self.get_anchor_fn = get_anchor_fn
33         self.handover_fn = handover_fn
34         self.available_zones = ["0.0.0.0:8888", "0.0.0.0:8889"]
```

```
36 def fit(
37     self, ins: FitIns,
38 ) -> FitRes:
39     """Define the operation before and after client fit the local model."""
40     if ins.config.contains(TRANSITION):
41         if ins.config[TRANSITION]:
42             idx = self.available_zones.index(self.get_anchor_fn())
43             idx = (idx + 1) % len(self.available_zones)
44             handover = True
45             del ins.config[TRANSITION]
46         else:
47             handover = False
48
49     res = self.trainer.fit(ins)
50
51     if handover:
52         log(INFO, "Client tries handover.")
53         self.handover_fn(self.available_zones[idx])
54
55     return res
```

如何讓 server 和 client 間使用加密通訊

可以在啟動 master/zone/client 時加上
certificates parameters 讓 grpc server
和 client 間的連線使用SSL

daisyfl/zone/app.py

```
56 def start_zone(  
57     *,  
58     parent_address: str = "",  
59     server_address: str = DEFAULT_SERVER_ADDRESS,  
60     grpc_max_message_length: int = GRPC_MAX_MESSAGE_LENGTH,  
61     uplink_certificates: Optional[bytes] = None,  
62     downlink_certificates: Optional[Tuple[bytes, bytes, bytes]] = None,  
63 ) -> None:
```

daisyfl/client/app.py

```
79 def start_client_numpy(  
80     *,  
81     server_address: str,  
82     parent_address: str,  
83     numpy_trainer: NumPyTrainer,  
84     grpc_max_message_length: int = GRPC_MAX_MESSAGE_LENGTH,  
85     uplink_certificates: Optional[bytes] = None,  
86     # downlink_certificates: Optional[Tuple[bytes, bytes, bytes]] = None,  
87     metadata: Tuple = None,  
88 ) -> None:
```

如何實作 secure aggregation

可參考

examples/06_secure_aggregation_pytorch、
daisyfl/operator/sec_agg、
daisyfl/strategy/sec_agg.py

server logic 可在一次 fit 內傳多次 fit
instruction 給 client 並收回多次 fit
response，透過給 FitIns.config 某個 key 的
不同值來和 client 進行不同的交流

daisyfl/operator/sec_agg/sec_agg_client_logic.py

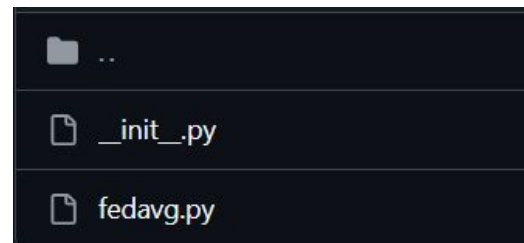
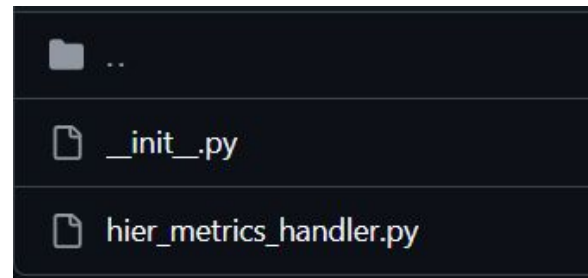
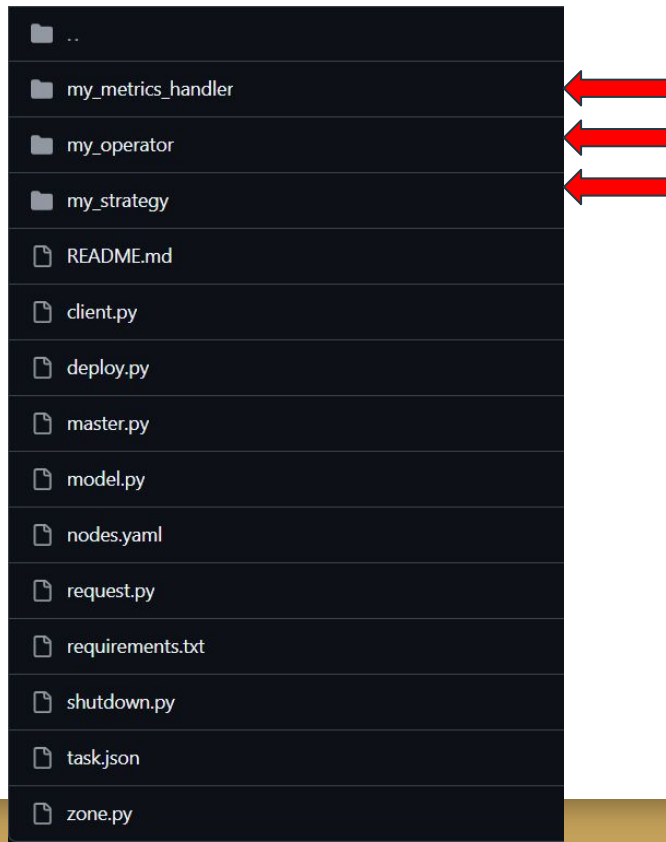
```
48 class SecAggClientLogic(ClientLogic):
49     |
50     |
51     def init(self, trainer, get_anchor_fn: Callable, handover_fn: Callable) -> None:
52         self.trainer = trainer
53         self.get_anchor_fn = get_anchor_fn
54         self.handover_fn = handover_fn
55     ~
56     def fit(
57         self, ins: FitIns,
58     ) -> FitRes:
59         """Define the operation before and after client fit the local model."""
60         stage = Proto(ins.config[PROTO_KEY])
61         if stage == Proto.SETUP:
62             return setup_param(self, ins)
63         elif stage == Proto.ASK_KEYS:
64             return ask_keys(self, ins)
65         elif stage == Proto.SHARE_KEYS:
66             return share_keys(self, ins)
67         elif stage == Proto.ASK_VECTORS:
68             return ask_vectors(self, ins)
69         elif stage == Proto.UNMASK_VECTORS:
70             return unmask_vectors(self, ins)
71         else:
72             raise ValueError("Invalid stage Proto received")
73
```



END



A. Make your own OPERATORS, STRATEGIES, and METRICS_HANDLERS



OPERATORS

```
class MyServerLogic(ServerLogic):
    """Base server (Zone or Master) operational logic defenition."""

    def __init__(self,
        communicator: Communicator,
        strategy: Strategy,
        metrics_handler: MetricsHandler,
    ) -> None:
        self.communicator: Communicator = communicator
        self.strategy: Strategy = strategy
        self.metrics_handler: MetricsHandler = metrics_handler

    def fit_round(
        self,
        parameters: Parameters,
        task: Task,
    ) -> Optional[
        Tuple[Optional[Parameters], Optional[Report]]
    ]:
```

```
    def evaluate_round(
        self,
        parameters: Parameters,
        task: Task,
    ) -> Optional[Report]:
        """Validate current global model on a number of clients."""
```

OPERATORS

```
class MyClientLogic(ClientLogic):
    """Base Daisy Client operational logic definition."""

    def __init__(self, trainer, get_anchor_fn: Callable, handover_fn: Callable,) -> None:
        self.trainer = trainer
        self.get_anchor_fn = get_anchor_fn
        self.handover_fn = handover_fn

    def fit(
        self, ins: FitIns,
    ) -> FitRes:
        """Define the operation before and after client fit the local model."""
        log(INFO, "=====.")
        log(INFO, "Welcome to developing your own module.")
        log(INFO, "You are using the customized module if you see this message.")
        log(INFO, "Client will fit the model.")
        log(INFO, "=====.")
        return self.trainer.fit(ins)

    def evaluate(
        self, ins: EvaluateIns,
    ) -> EvaluateRes:
        """Define the operation before and after client evaluate the local model."""
        log(INFO, "=====.")
        log(INFO, "Welcome to developing your own module.")
        log(INFO, "You are using the customized module if you see this message.")
        log(INFO, "Client will evaluate the model.")
        log(INFO, "=====.")
        return self.trainer.evaluate(ins)
```


STRATEGIES

```
class MyFedAvg(Strategy):
    """Configurable FedAvg strategy implementation."""
    def __init__(
        self,
        client_manager: ClientManager,
        num_clients_fit: int = 2,
        num_clients_evaluate: int = 2,
        min_results_fit: int = 2,
        min_results_evaluate: int = 2,
    ) -> None:
        """Federated Averaging strategy.

    def configure_fit(
        self,
        parameters: Parameters,
        config: Dict,
        **kwargs,
    ) -> List[Tuple[ClientProxy, FitIns]]:
        """Configure the next round of fitting."""
```

```
def configure_evaluate(
    self,
    parameters: Parameters,
    config: Dict,
    **kwargs,
) -> List[Tuple[ClientProxy, EvaluateIns]]:
    """Configure the next round of evaluation."""

def wait_fit(
    self, subtask_status: SubtaskStatus, **kwargs
) -> bool:
    """Wait for the termination condition of fitting."""

def wait_evaluate(
    self, subtask_status: SubtaskStatus, **kwargs
) -> bool:
    """Wait for the termination condition of evaluating."""

def aggregate_fit(
    self,
    results: List[Tuple[ClientProxy, FitRes]],
    **kwargs,
) -> Tuple[Optional[Parameters], Dict]:
    """Aggregate fit results using weighted average."""
```

STRATEGIES

```
def aggregate_evaluate(  
    self,  
    results: List[Tuple[ClientProxy, EvaluateRes]],  
    **kwargs,  
) -> Dict:  
    """Aggregate evaluation losses using weighted average."""
```

METRICS_HANDLERS

```
class MyMetricsHandler(MetricsHandler):
    """MetricsHandler class to cope with metrics update and exposure."""

    def __init__(
        self,
    ) -> None:
        log(DEBUG, "Start MetricsHandler.")
        super().__init__()
        self.acc: List = []
        self.loss: List = []
        self.fit_data_samples: List = []
        self.evaluate_data_samples: List = []

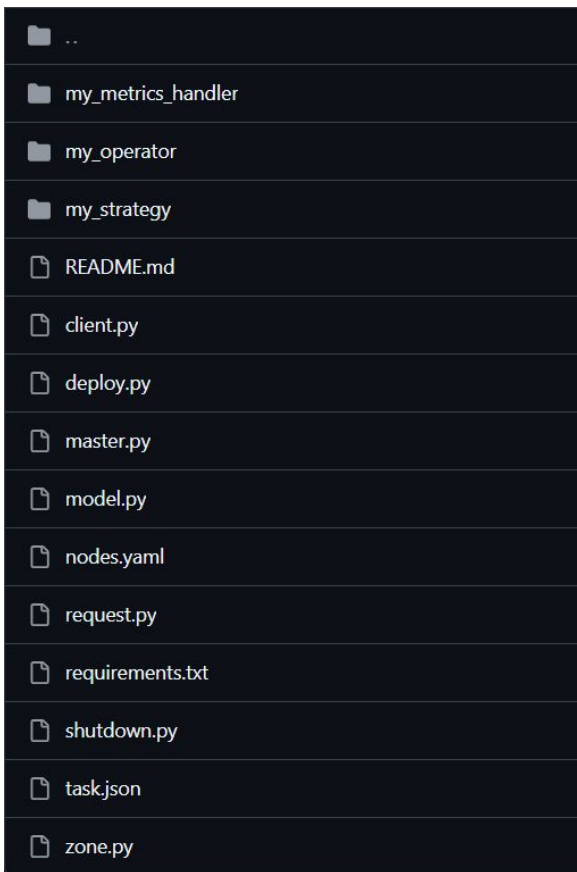
    def update_metrics_fit(self, config: Dict):
        """Update fitting metrics."""

    def update_metrics_evaluate(self, config: Dict):
        """Update evaluation metrics."""
```

```
def get_metrics(self,):
    """Expose metrics through API gateway."""
    # DEFAULT: expose metrics to prometheus.
```

```
def __repr__(self) -> str:
    """Print metrics."""
```

B. Change “task.json” file



```
{
  "TID": "customized_fl_algorithm_demo",
  "NUM_ROUNDS": 2,
  "ZONE_COMM_FREQUENCY": 2,
  "EVALUATE_INTERVAL": 1,
  "EVALUATE_INIT_MODEL_MASTER": true,
  "MIN_WAITING_TIME_MASTER": 10,
  "MIN_WAITING_TIME_ZONE": 5,
  "MODEL_PATH": "model.npy",
  "OPERATORS": {
    "MASTER_SERVER_OPERATOR": ["my_operator.server_logic", "MyServerLogic"],
    "ZONE_SERVER_OPERATOR": ["my_operator.server_logic", "MyServerLogic"],
    "CLIENT_OPERATOR": ["my_operator.client_logic", "MyClientLogic"]
  },
  "STRATEGIES": {
    "MASTER_STRATEGY": ["my_strategy.fedavg", "MyFedAvg"],
    "ZONE_STRATEGY": ["my_strategy.fedavg", "MyFedAvg"]
  },
  "METRICS_HANDLERS": {
    "MASTER_METRICS_HANDLER": ["my_metrics_handler.hier_metrics_handler", "MyMetricsHandler"],
    "ZONE_METRICS_HANDLER": ["my_metrics_handler.hier_metrics_handler", "MyMetricsHandler"]
  },
  "SAVE_MODEL": true
}
```